

CSC240DynamicArrayQueue/QueType.h

```
1  #ifndef QUEUETYPE_H
2  #define QUEUETYPE_H
3  #include <iostream>
4  #include <cmath>
5  using namespace std;
6  class FullQueue
7  {};
8  class EmptyQueue
9  {};
10 typedef char ItemType;
11 class QueType
12 {
13 public:
14     friend ostream& operator<<(ostream& out, const QueType& q);
15
16     QueType();
17     // Class constructor.
18     // Because there is a default constructor, the precondition
19     // that the queue has been initialized is omitted.
20     QueType(int max);
21     // Parameterized class constructor.
22     ~QueType();
23     // Class destructor.
24     void MakeEmpty();
25     // Function: Initializes the queue to an empty state.
26     // Post: Queue is empty.
27     bool IsEmpty() const;
28     // Function: Determines whether the queue is empty.
29     // Post: Function value = (queue is empty)
30     bool IsFull() const;
31     // Function: Determines whether the queue is full.
32     // Post: Function value = (queue is full)
33     void Enqueue(ItemType newItem);
34     // Function: Adds newItem to the rear of the queue.
35     // Post: If (queue is full) FullQueue exception is thrown
36     //        else newItem is at rear of queue.
37     void Dequeue(ItemType& item);
38     // Function: Removes front item from the queue and returns it in item.
39     // Post: If (queue is empty) EmptyQueue exception is thrown
40     //        and item is undefined
41     //        else front element has been removed from queue and
42     //        item is a copy of removed element.
43
44     ItemType DequeueRear(); //IT 3/4/2020
45     //Function: Remove and return the rear item of a queue
46
47     void evacuate(int num);
```

```
48 private:
49     int front;
50     int rear;
51     ItemType* items;
52     int maxQue;
53 };
54
55
56 QueType::QueType(int max)
57 // Parameterized class constructor
58 // Post: maxQue, front, and rear have been initialized.
59 //     The array to hold the queue elements has been dynamically
60 //     allocated.
61 {
62     maxQue = max + 1;
63     front = maxQue - 1;
64     rear = maxQue - 1;
65     items = new ItemType[maxQue];
66 }
67 QueType::QueType()           // Default class constructor
68 // Post: maxQue, front, and rear have been initialized.
69 //     The array to hold the queue elements has been dynamically
70 //     allocated.
71 {
72     maxQue = 501;
73     front = maxQue - 1;
74     rear = maxQue - 1;
75     items = new ItemType[maxQue];
76 }
77 QueType::~QueType()          // Class destructor
78 {
79     delete [] items;
80 }
81
82 void QueType::MakeEmpty()
83 // Post: front and rear have been reset to the empty state.
84 {
85     front = maxQue - 1;
86     rear = maxQue - 1;
87 }
88
89 bool QueType::IsEmpty() const
90 // Returns true if the queue is empty; false otherwise.
91 {
92     return (rear == front);
93 }
94 #endif
95
96 bool QueType::IsFull() const
```

```
97 // Returns true if the queue is full; false otherwise.
98 {
99     return ((rear + 1) % maxQue == front);
100 }
101
102 void QueueType::Enqueue(ItemType newItem)
103 // Post: If (queue is not full) newItem is at the rear of the queue;
104 //       otherwise a FullQueue exception is thrown.
105 {
106     if (IsFull())
107         throw FullQueue();
108     else
109     {
110         rear = (rear + 1) % maxQue;
111         items[rear] = newItem;
112     }
113 }
114
115 void QueueType::Dequeue(ItemType& item)
116 // Post: If (queue is not empty) the front of the queue has been
117 //       removed and a copy returned in item;
118 //       otherwise a EmptyQueue exception has been thrown.
119 {
120     if (IsEmpty())
121         throw EmptyQueue();
122     else
123     {
124         front = (front + 1) % maxQue;
125         item = items[front];
126     }
127 }
128
129 ItemType QueueType::DequeueRear(){
130     ItemType item;
131     if (IsEmpty())
132         throw EmptyQueue();
133     else
134     {
135         item = items[rear];
136         rear = (rear - 1 + maxQue) % maxQue;
137     }
138     return item;
139 }
140
141
142 void QueueType::evacuate(int num){
143     cout << "rear = " << rear << " front: " << front << endl;
144     for(int i = 0; i < num; i++){
145         if(IsEmpty()){
```

```
146         throw EmptyQueue();
147     }
148     DequeueRear();
149 }
150 }
151
152
153
154 ostream& operator<<(ostream& out, const QueueType& q){
155     if((q.rear - q.front + q.maxQue) % q.maxQue == 0){
156         out << "Empty queue." << endl;
157     }
158     else{
159         out << "Front: ";
160         for(int i = 0; i < (q.rear - q.front + q.maxQue) % q.maxQue; i++){
161             out << q.items[(q.front + 1 + i) % q.maxQue] << " ";
162         }
163         out << endl;
164     }
165     return out;
166 }
167
168
169
170
171
172
173
174
175
176
177
178
```